

UMC

Curso: Engenharia do Software

Turma: 5B

Professor: Pedro Toledo

Objetivo: Compreender o LSP do SOLID e implementá-lo corretamente.

Introdução ao SOLID

O SOLID consiste em um conjunto de cinco princípios fundamentais para a programação orientada a objetos, visando um código mais flexível, escalável e fácil de manter.

Os princípios são:

1. Single Responsibility Principle (SRP) – Cada classe deve ter apenas uma razão para mudar.
2. Open-Closed Principle (OCP) – Classes devem estar abertas para extensão, mas fechadas para modificação.
3. Liskov Substitution Principle (LSP) – Subtipos devem ser substituíveis por seus tipos base sem alterar o comportamento esperado do sistema.
4. Interface Segregation Principle (ISP) – Interfaces específicas devem ser preferidas a interfaces genéricas.
5. Dependency Inversion Principle (DIP) – Os módulos de alto nível não devem depender de módulos de baixo nível; ambos devem depender de abstrações.

Nesta aula, o enfoque será para o LSP.



Princípio de Substituição de Liskov (LSP)

“Se S é um subtipo de T, então objetos de tipo T em um programa podem ser substituídos por objetos de tipo S sem alterar as propriedades desejáveis desse programa”

Barbara Liskov

Proposto por Barbara Liskov, este princípio enfatiza que classes derivadas (subclasses) devem ser capazes de substituir suas classes base sem causar efeitos colaterais no funcionamento do programa. Isso significa que:

- Métodos herdados não devem alterar a lógica esperada.
- Métodos da classe base devem ser implementados corretamente nas subclasses.

- A substituição não deve quebrar nenhuma funcionalidade do sistema.

Diretrizes para o LSP

- **Pré-condições** não podem ser fortalecidas na classe base, isto é, a subclasse não pode exigir mais regras do que a classe base exige. Dica: muita atenção ao sobrescrever métodos.
- **Pós-condições** não podem ser enfraquecidas nas subclasses, isto é, as subclasses não podem reduzir regras fornecidas (garantias) pela classe base. Dica: muita atenção no tipo e no significado/comportamento do retorno dos métodos.
- **Invariantes** não podem ser enfraquecidas no subtipo, isto é, a subclasse não pode alterar as condições internas que a classe base mantinha constantes. Dica: muita atenção para não alterar regras que não podem variar.

Caso contrário, o código perde sua previsibilidade e introduz comportamentos inesperados que poderiam comprometer os resultados do código.

Um exemplo de Violação do Princípio de Liskov

Suponha que temos uma classe Produto, uma classe ProdutoFisico e uma ProdutoDigital e que as duas últimas são subclasses de Produto.

Analise os trechos de código a seguir.

```
public class Produto {  
    protected String nome;  
    protected double preco;  
    public Produto(String nome, double preco) {  
        this.nome = nome;  
        this.preco = preco;  
    }  
    public double calcularFrete() {  
        return this.preco * 0.1; // 10% do preço  
    }  
}
```

```
public class ProdutoFisico extends Produto{
```

```

public ProdutoFisico(String nome, double preco) {
    super(nome, preco);
}
@Override
public double calcularFrete() {
    return this.preco * 0.15; // 15% do preço
}
}

public class ProdutoDigital extends Produto{
    public ProdutoDigital(String nome, double preco) {
        super(nome, preco);
    }
    @Override
    public double calcularFrete() {
        throw new UnsupportedOperationException("Produto digital não tem frete!");
    }
}

```

No exemplo acima, ProdutoDigital não deveria herdar o método calcularFrete(), considerando que um produto digital não possui frete. Isso quebra o princípio de Liskov, pois ao substituir Produto ou ProdutoFisico por ProdutoDigital, o sistema pode falhar.

Geralmente, o problema ocorre em razão de se criar abstrações excessivamente amplas e no exemplo acima, o problema surgiu porque um ProdutoDigital não possui um método para calcular o frete, mas herdou essa funcionalidade.

Implementação de acordo com diretrizes do LSP

A solução correta seria criar uma interface ProdutoFisico e outra para ProdutoDigital e implementar o cálculo de frete somente na classe apropriada.

(Verificar exemplo de código no projeto PP_SOLID_Liskov.zip)

Desafio: Suponha que seu sistema possui uma funcionalidade de autenticação do usuário (com login e senha) em seu projeto de CRUD e implemente uma funcionalidade para retornar um recurso externo ao projeto caso o usuário forneça credenciais inválidas.